Supplemental Information for "PoolParty: Declartive design of commplex oligonucleotide libraries"

[Author Names]

January 1, 2026

Contents

1 Introduction **2**
1.1 Example 1: Fixed and Random Mode Operations 2
1.2 Example 2: Sequential Mode (Cartesian Product + Stack Enumeration) 4

2 StateCounter Architecture **5**
2.1 Core Concepts 5
2.2 The Manager Context 5

3 StateCounter Operations **6**

4 PoolParty Architecture **6**
4.1 Pool Composition 6
4.2 Generation Modes 6

5 PoolParty Operations **6**

6 Integration Between PoolParty and StateCounter **7**

7 Code Examples **7**
7.1 StateCounter Example 7
7.2 PoolParty Example 8
7.3 Advanced Example: Combining Operations 8

1 Introduction

PoolParty makes it easy to design complex libraries involving barcodes, mutagenesis, insertion and deletion scanning, and other common operations.

The core idea behind PoolPart is that library construction should be *declarative* rather than *imperative*. Rather than explicitly constructing each sequence in the library, users specify high-level operations that, when carried out in series, generated the desired library. After laying out these rules, users simiply specify how many sequence to generate and poolparty takes care of the rest. This approach separates the *what* (library specification) from the *how* (enumeration or sampling),

Poolparty has two core types of objects: Pools and Operations. A Pool object represents a yet-to-be-realized set of sequences. An Operation object represents a computation that yields a Pool object, and can take zero, one, or multiple Pool objects as input. To create a sequence library, users first specify the rules for making the library by chaining together a series of Operations, the result of which is a Pool boject. Users then call on that pool to generate the number of desired sequences.

The following two examples illustrate PoolParty's core capabilities.

1.1 Example 1: Fixed and Random Mode Operations

Each component Operation of the computation graph can be in one of four modes: `'fixed'`, `'random'`, `'sequential'`, or `'hybrid'`. Fixed mode operations are deterministic: for a given input they always produce the same output. Random mode operation, on the other hand, will generally produce a different output each time they are called due to them having an internal state that changes unpredictably.

Here is an example of PoolParty code that uses `'random'` and `'fixed'` operations to generate the type of construct commonly used in massively parallel reporter assays (MPRAs). First the core sequence elemenets are defined as simple python strings: `wt_seq` is a regulatory sequence of iterest, `adapter_5_seq` is a constant-sequence adapter, and `adapter_3_iupac` is an adapter containing degerate nucleotides that will be used as a barcode. Next, the `pp.mutagenize()` operation is called on `wt_seq`, generating a pool `mut_pool` comprising randomly mutated versions of `wt_seq`. Next, the `pp.from_iupac_motif()` operation is called on `adapter_3_iupac`, generating a pool `adapter_3_pool` in which the `N` characters have been replaced by randomly selected DNA nucleotides. Next, the `pp.join` operation deterministically joins the 3ádaptor sequence, mutagenized pool, and barcode-containing 5ádaptor sequence into a final `oligo_pool_1`. Finally, the `pp.generate_seqs()` method is called with the desired number of sequence. This produces a pandas dataframe `df` that lists each sequence along with a "design card", i.e. a series of fields that describes the decisions each operation made in order to arrive at each resulting sequence.

```
import poolparty as pp

# Mark changes by changing case of affected bases
pp.set_default('mark_changes',True)

# Define component sequences
wt_seq  = 'ATCGATCGATATCGATCGAT'         # 10 nt wild-type
ad5_seq = 'GAAC'                         # 3 nt 5' adapter
ad3_seq = 'TGGANNNNNCCAG'                # 3 + 5N barcode + 3
```

```python
# Create pool of randomly mutagenized wt_seq sequences
mut_pool = pp.mutagenize(wt_seq, num_mutations=3)

# Create pool of 3' adaptors with random barcodes
ad3_pool = pp.from_iupac_motif(ad3_seq, mode='random')

# Join into final oligo pool
oligo_pool = pp.join([ad5_seq, mut_pool, ad3_pool], spacer_str=' ')

# Sample 5 sequences
df = oligo_pool.generate_seqs(num_seqs=5, seed=42)
print(df['seq'])
```

Key points:

- `pp.set_default('mark_changes',True)` sets the default option to mark all changes (mutations) in the generated sequences, typically displaying them in a different case for easy identification.
- `pp.mutagenize()` generates a pool of sequences containing different combinations of mutations at within a specified wild-type sequence. Here we instruct poolparty to introduce 3 mutations into each sequence. This operation runs in random mode by default, which means that the specific mutations that each sequence recieves is random.
- `pp.from_iupac_motif()` generates a pool of sequences derived from an IUPAC sequence, i.e., a DNA sequence in which special characters (such as Ń) signify degenerate bases. This too runs in random mode by default.
- `pp.join()` joins together multiple sequences/Pools. It has no variable internal state so always runs in fixed mode. It also takes an argment, `spacer_str`, which we use here to add a space between the mutagenized region and adaptor sequences.
- `generate_seqs(num_seqs=N, seed)` instructs `oligo_pool` to construct N sequences using the specified random seed. It retuns a Pandas dataframe, here called `df`. The first column is named `'seq'` and contains the resulting sequences:

```
0    GAAC AgCGATCGATATCatTCGAT TGGAatatgCCAG
1    GAAC AgCGATCGATtTCGATCGAc TGGAagaatCCAG
2    GAAC ATCGATCGtTtTCGAcCGAT TGGAcgctaCCAG
3    GAAC ATCGATCGcTgTCGtTCGAT TGGAacaacCCAG
4    GAAC ATCGAgCGATATCGAgCGgT TGGAacaccCCAG
Name: seq, dtype: object
```

- The other columns of the dataframe contain design card information, i.e., the information that was used to create each sequence variant (Note: columns have been pruned and renamed for simplicity).

	seq	iupac_state	positions	wt_chars	mut_chars
0	GAAC AgCGATCGATATCatTCGAT TGGAatatgCCAG	206	(1, 13, 14)	(T, G, A)	(g, a, t)
1	GAAC AgCGATCGATtTCGATCGAc TGGAagaatCCAG	131	(1, 10, 19)	(T, A, T)	(g, t, c)
2	GAAC ATCGATCGtTtTCGAcCGAT TGGAcgctaCCAG	412	(8, 10, 15)	(A, A, T)	(t, t, c)
3	GAAC ATCGATCGcTgTCGtTCGAT TGGAacaacCCAG	65	(8, 10, 14)	(A, A, A)	(c, g, t)
4	GAAC ATCGAgCGATATCGAgCGgT TGGAacaccCCAG	69	(5, 15, 18)	(T, T, A)	(g, g, g)

- During the execution of `oligo_pool.generate_seqs()`, each sequence is computed one-at-a-time using a computation tree that can be visualized by running `oligo_pool.print_tree()`:

```
pool[4] (pool, n=1)
└── op[4]:join [op=join, mode=fixed, n=1]
    ├── pool[3] (pool, n=1)
    │   └── op[3]:from_seq [op=from_seq, mode=fixed, n=1]
    ├── pool[1] (pool, n=1)
    │   └── op[1]:mutagenize [op=mutagenize, mode=random, n=1]
    │       └── pool[0] (pool, n=1)
    │           └── op[0]:from_seq [op=from_seq, mode=fixed, n=1]
    └── pool[2] (pool, n=1)
        └── op[2]:from_iupac_motif [op=from_iupac_motif, mode=random, n=1]
```

1.2 Example 2: Sequential Mode (Cartesian Product + Stack Enumeration)

This example demonstrates sequential enumeration of a combinatorial library. We create a replacement scan (with two insert variants at four positions) and a deletion scan, then stack them to enumerate all 12 combinations.

```python
# Create insert pool with two variants
ins_pool = pp.from_seqs(['AAAAA', 'TTTTT'], mode='sequential')

# Replacement scan at step=5 intervals
rep_scan_pool = pp.replacement_scan(
    wt_seq, ins_pool,
    positions=range(0, 20, 5),  # [0, 5, 10, 15]
    mode='sequential',
    mark_inserts=True
)

# Deletion scan at same positions
del_scan_pool = pp.deletion_scan(
    wt_seq,
    deletion_length=5,
    positions=range(0, 20, 5),
    mode='sequential',
    mark_deletions=True
)

# Stack replacement scan and deletion scan (union of states)
```

```
combo_scan_pool = pp.stack([rep_scan_pool, del_scan_pool])

# Join into final oligo (reusing adapter_3_pool from above)
oligo_pool_2 = pp.join([adapter_5_seq, combo_scan_pool, adapter_3_pool],
↪  spacer_str='.')

# Enumerate all 12 combinations: (2 inserts x 4 positions) + (4 deletions)
df = oligo_pool_2.generate_seqs(num_complete_iterations=1, seed=42)
print(df['seq'].tolist())
```

Example output (12 sequences, all same length, inserts in UPPERCASE):

```
GAATTC.AAAAAcgatcgatcgatcg.GGATCC.agtcgtatca.CTGCAG  # AAAAA at pos 0
GAATTC.atcgAAAAAcgatcgatcg.GGATCC.tcagctgatc.CTGCAG  # AAAAA at pos 5
GAATTC.atcgatcgAAAAAatcgatcg.GGATCC.gcatgctagc.CTGCAG  # AAAAA at pos 10
GAATTC.atcgatcgatcgAAAAAcg.GGATCC.atcgatcgat.CTGCAG  # AAAAA at pos 15
GAATTC.TTTTTcgatcgatcgatcg.GGATCC.tgcatgcatg.CTGCAG  # TTTTT at pos 0
GAATTC.atcgTTTTTcgatcgatcg.GGATCC.cgatcgatcg.CTGCAG  # TTTTT at pos 5
GAATTC.atcgatcgTTTTTatcgatcg.GGATCC.atgcatgcat.CTGCAG  # TTTTT at pos 10
GAATTC.atcgatcgatcgTTTTTcg.GGATCC.gctagctagc.CTGCAG  # TTTTT at pos 15
GAATTC.-----cgatcgatcgatcg.GGATCC.agtcgtatca.CTGCAG  # del at pos 0
GAATTC.atcg-----cgatcgatcg.GGATCC.tcagctgatc.CTGCAG  # del at pos 5
GAATTC.atcgatcg-----atcgatcg.GGATCC.gcatgctagc.CTGCAG  # del at pos 10
GAATTC.atcgatcgatcg-----cg.GGATCC.atcgatcgat.CTGCAG  # del at pos 15
```

Key points:

- `mode='sequential'` enables deterministic enumeration of all states
- Cartesian product: insert variants $\times$ scan positions gives $2 \times 4 = 8$ replacement states
- `pp.stack()` creates a union of states ($8 + 4 = 12$ total)
- `mark_inserts=True` highlights inserted sequences in UPPERCASE
- `generate_seqs(num_complete_iterations=N)` iterates through all states N times

The remainder of this document describes the architecture and operations of PoolParty and its underlying StateCounter library in detail.

2 StateCounter Architecture

[Write description of StateCounter's core architecture, including the Counter class, Manager context, and unidirectional state propagation mechanism.]

2.1 Core Concepts

[Describe core concepts: leaf counters, derived counters, state propagation, and conflict detection.]

2.2 The Manager Context

[Explain the Manager context manager and its role in the StateCounter system.]

3 StateCounter Operations

[Write overview of StateCounter operations and how they combine counters to create complex enumeration patterns.]

Table 1: StateCounter Operations

Operation	Syntax	Description
Product	`A * B`	Cartesian product of two counters
Stack	`stack([A, B])`	Union/sum of counters (one active at a time)
Slice	`A[start:stop:step]`	Subset of counter states
Repeat	`A * n`	Repeat counter n times
Shuffle	`shuffle_counter(A)`	Randomly permute state order
Sample	`sample_counter(A, k)`	Random sample of k states
Synchronize	`synchronize_counters(A, B)`	Lock counters to same state
Split	`split_counter(A, sizes)`	Split counter into parts
Interleave	`interleave_counters([A, B])`	Interleave states from counters
Passthrough	`passthrough(A)`	Identity operation (for debugging)

4 PoolParty Architecture

[Write description of PoolParty's core architecture, including the Pool class, lazy evaluation, and how Pools compose to define oligonucleotide libraries.]

4.1 Pool Composition

[Describe how Pool objects can be composed using string-like operations.]

4.2 Generation Modes

[Explain the different sequence generation modes available in PoolParty.]

5 PoolParty Operations

[Write overview of PoolParty operations and how they enable different library design patterns.]

Table 2: PoolParty Operations

Operation	Category	Description
`from_seq`	Creation	Create pool from a single sequence
`from_seqs`	Creation	Create pool from multiple sequences
`from_iupac_motif`	Creation	Create pool from IUPAC ambiguity codes
`from_prob_motif`	Creation	Create pool from position weight matrix
`get_kmers`	Creation	Generate all k-mers from alphabet

(continued on next page)

(continued from previous page)

Operation	Category	Description
`deletion_scan`	Scanning	Systematic single-position deletions
`insertion_scan`	Scanning	Systematic single-position insertions
`replacement_scan`	Scanning	Systematic single-position replacements
`breakpoint_scan`	Scanning	Scan for breakpoint positions
`marker_scan`	Marker	Scan for marker positions
`marker_multiscan`	Marker	Scan multiple markers
`replace_marker`	Marker	Replace marker with sequence
`mutagenize`	Mutagenesis	Apply mutations to sequence
`mutagenize_orf`	Mutagenesis	Apply codon-aware mutations to ORF
`join`	Manipulation	Join multiple pools
`stack`	Manipulation	Stack pools (one active at a time)
`repeat`	Manipulation	Repeat pool sequence
`seq_slice`	Manipulation	Slice sequence positions
`seq_shuffle`	Manipulation	Shuffle sequence order
`reverse_complement`	Manipulation	Reverse complement sequence
`state_slice`	State	Slice pool states
`state_shuffle`	State	Shuffle pool states
`state_sample`	State	Sample pool states
`sync`	State	Synchronize pool states

6 Integration Between PoolParty and StateCounter

[Write description of how PoolParty uses StateCounter internally, including how Pool operations wrap Counter operations and how state propagation enables lazy sequence generation.]

7 Code Examples

7.1 StateCounter Example

The following example demonstrates basic StateCounter usage with the product operation:

```
from statecounter import Counter, Manager

with Manager():
    # Create leaf counters
    A = Counter(num_states=2, name='A')
    B = Counter(num_states=3, name='B')

    # Combine with product (Cartesian product)
    C = A * B  # 6 states total

    # Iterate and see parent states update
```

```
    for state in C:
        print(f"C={state}, A={A.state}, B={B.state}")
```

To create a counter, use `Counter(num_states=n, name='X')`.

7.2 PoolParty Example

The following example demonstrates creating a mutagenesis library with barcodes:

```
from poolparty import Pool

# Define the DNA sequence
seq = "ATCGATCGATCGATCGATCG"

# Create a pool with all single deletions
pool = Pool.from_seq(seq).deletion_scan()

# Add a barcode region
barcode = Pool.get_kmers(length=10, alphabet='ACGT')

# Concatenate to create final library
library = pool + barcode

# Generate sequences
seqs_df = library.generate_library(
    combinatorially_complete_pools=[pool],
    num_complete_iterations=5
)
```

7.3 Advanced Example: Combining Operations

```
# Example: Create a library with multiple scanning operations
# [Add your advanced example here]
pass
```